Lending Club P2P Loan Default Prediction: Reproduction and Improvements

UE24CS352A Machine Learning Mini-Project · Team A-15, Section A · Problem Statement 39
Aman Priyansh (PES1UG24AM030) · Addanki Harsha Rajeswari (PES1UG24AM014)

1. Problem statement

Peer-to-peer (P2P) lending platforms such as LendingClub let individuals lend to borrowers directly, so investors carry the credit risk themselves. We reproduce and then improve the Stanford CS229 project of Li and Han (2018). It builds (i) **classifiers** that predict whether a loan will be fully paid or default, (ii) **regressors** that predict a loan's net annualized return (NAR), and (iii) a loan-selection strategy that invests only in loans whose predicted NAR exceeds a cut-off. The paper reports 0.89 weighted precision and recall, and a strategy earning about 15% a year on about 1.7% of loans.

2. Dataset

LendingClub's own download page no longer exists, so we use the public Kaggle copy (**wordsforthewise/lending-club**, 2.26 million accepted loans, 2007–2018). We apply the paper's rules: loans issued 2012–2015 with a final status (Fully Paid, Charged Off, Default). This gives **786,820 loans** (paper: 745,529, because the copy is newer and more loans have finished), of which **18.6% default** (paper: 19%). After the authors' cleaning (mean, zero and max×5 imputation, one-hot encoding, standardisation) there are **1,099 features** (paper: 1,097). The regression label is the authors' NAR: (total paid ÷ amount funded) raised to 365 ÷ (days from issue to last payment + 30).

3. Approach

Baseline. We reproduce every model in the paper with the authors' settings, taken from their released code where it exists: Logistic Regression (balanced class weights), a Neural Network (10-10-5-3, sigmoid), and a Random Forest (200 trees, depth 30, 50 features per split) for classification; Linear, Ridge, a Neural Network (20-10-5-3, ReLU) and Random Forests of depth 4/8/10 for regression; and the authors' cut-off rule for the strategy.

Problems found and improvements. (1) **Data leakage**: last_fico_range_high/low are the borrower's credit score after the loan ended, which no lender has at decision time. out_prncp/out_prncp_inv are always 0 for finished loans. We remove all four. (2) The **random split** mixes years, so we train on 2012–2014 and test on 2015 loans. (3) The paper had **no validation set and no tuning**, so we hold out 15% of training data for early stopping and a 12-trial random search. (4) The paper never tried **gradient boosting**, so we add XGBoost. (5) Scores at a single cut-off are complemented by **ROC-AUC** and **PR-AUC**. Changes are applied one at a time (ablation): **S1** = paper setup, **S2** = leak removed, **S3** = leak removed + future test.

4. Implementation overview

- Two Google Colab notebooks (Python, scikit-learn, XGBoost), run on an A100 GPU runtime: 01_Baseline_Reproduction (about 20 min) and 02_Improvements (about 15–20 min). Both download the data themselves.
- Missing pieces rebuilt: the authors' data_clean.csv (not in their zip) from the paper's rules, and the Neural Network classifier (no code file) from the paper's settings. A fixed seed (0) makes the 70/30 split reproducible.
- The CPU Random Forest regressor is replaced by XGBoost's random-forest mode on the GPU (same 100 trees, depths and all features per split, with 128-bin histogram splits). Every deviation is listed in the README.
- A Gradio demo in the improvements notebook shows the honest model's default probability for any 2015 test loan, next to the real outcome.

5. Results

Baseline (test set)	Paper	Ours
Logistic Regression / Neural Net / Random Forest, weighted F1	0.88 / 0.89 / 0.89	0.88 / 0.88 / 0.89
Ridge / Neural Net / RF depth 10, regression R^2	0.238 / 0.306 / 0.315	0.217 / 0.252 / 0.281
Investment strategy	~1.7% of loans, ~15% a year	1.34% of loans, 15.2% a year

Table 1. The baseline is reproduced: classification within 0.01, regression within 0.02–0.05 R^2, and the same strategy return.

Ablation (test set)	Log. Reg.	Rand. Forest	XGBoost	XGB tuned	Strategy: top 1.7% return
S1: paper setup	0.935	0.934	0.941	–	15.1% (RF)
S2: leak removed	0.721	0.721	0.733	–	–
S3: leak removed + test on 2015	0.733	0.731	0.743	**0.744**	3.8% (XGB); all loans −1.6%

Table 2. ROC-AUC for each model and setting, and the true yearly return of the strategy's top 1.7% of loans.

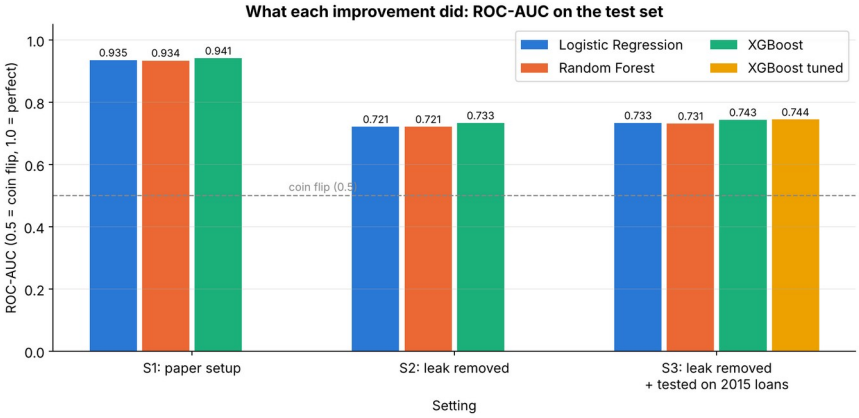


Figure 1. Removing the leaky columns drops every model from about 0.94 to about 0.72 ROC-AUC; testing on future loans does not reduce it further.

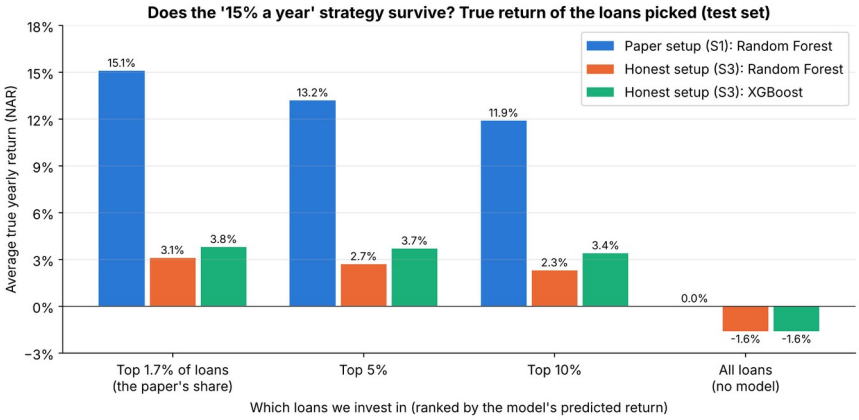


Figure 2. True yearly return of the loans each model picks: 15.1% in the paper's setup, 3.8% under the honest setup.

6. Conclusions

- We reproduced the paper's results after rebuilding its missing data file and Neural Network code.
- Most of the paper's accuracy comes from **data leakage**: the borrower's credit score after the loan ended. Removing it lowers ROC-AUC from about 0.94 to about 0.72 for every model.
- Honest models still generalise to future loans: XGBoost is best in every setting (tuned ROC-AUC 0.744). Tuning adds little, so the limit is the information in the application data.
- The paper's 15% strategy falls to about **3.8% a year** under honest conditions. That still beats investing in every loan (−1.6%) by about 5 points. The 2015 test loans default more often (20.2%, because the data ends in 2018), so this estimate is slightly pessimistic.